UNIVERSIDAD POLITÉCNICA
BICENTENARIO

Control ON-OFF de Temperatura.

Presenta:

Francisco Emmanuel Hernández Camacho *24033418*

Miguel Ángel Ramírez Elías *24033552*

Diego Gomes Muñiz *24033476*

Yessenia Medrano Castillo *24033579*

Fernanda Jaqueline Ibarra Romero *24032986*

Asignatura:

Sistemas Embebidos

Profesor:

Hugo Antonio Méndez Guzmán

Silao, Gto. A 28 de septiembre del 2026

INDICE

Introducción **3**
Marco Teórico **4**
Control On-Off : **4**
Componentes Utilizados: **4**
Desarrollo **5**
Material requerido: **5**
Asignación de pines **5**
Resultados **7**
Observaciones y Conclusiones **8**
Observaciones: **8**
Conclusiones: **8**

Introducción

En el ámbito de los sistemas embebidos y la automatización industrial, el monitoreo y control de variables físicas como la temperatura es una tarea fundamental. Muchos procesos industriales, agrícolas y domésticos requieren mantener un ambiente controlado dentro de rangos específicos para garantizar la integridad de los productos, la eficiencia energética o la seguridad de un sistema. Para lograr esto, los microcontroladores actúan como el cerebro del sistema, leyendo señales de sensores y accionando mecanismos correctivos en tiempo real.

En esta práctica, se implementó un sistema de control On-Off de temperatura utilizando la tarjeta de desarrollo ESP32. El sistema consiste en una caja térmicamente cerrada donde se busca regular el ambiente mediante dos actuadores opuestos: un foco incandescente conectado a un dimmer (como fuente de calor) y un ventilador con celda Peltier (como fuente de enfriamiento). La temperatura interna se monitorea constantemente mediante un sensor digital DS18B20, y el estado del sistema, así como la lectura térmica, se visualizan en una pantalla LCD a través del protocolo I2C. Adicionalmente, se integraron botones pulsadores para interactuar con la configuración del sistema.

El objetivo principal de esta actividad es entender la lógica de control de lazo cerrado, aprender a integrar sensores digitales, pantallas de visualización y módulos de potencia (relés) con el ESP32, y establecer la relación entre los conceptos teóricos de control y su aplicación práctica en un sistema físico capaz de mantener una variable en un estado deseado. Esto sienta las bases para el desarrollo de sistemas de automatización más complejos.

Marco Teórico

En el ámbito de los sistemas embebidos y la automatización, el control de variables físicas como la temperatura es fundamental. Un sistema de control de lazo cerrado compara el valor medido de una variable (en este caso, la temperatura) con un valor deseado (setpoint) para tomar decisiones y accionar actuadores.

Control On-Off :

También conocido como control de dos posiciones o "bang-bang", es el tipo más simple de control. Consiste en encender o apagar un actuador cuando la variable cruza un umbral. Por ejemplo, si la temperatura desciende por debajo de un límite inferior, se enciende un calentador; si sube por encima de un límite superior, se enciende un ventilador o sistema de enfriamiento. Generalmente se utiliza una "histéresis" para evitar que el sistema se encienda y apague constantemente cuando la temperatura oscila cerca del punto de consigna.

Componentes Utilizados:

- **ESP32:** Microcontrolador de alto rendimiento con conectividad Wi-Fi y Bluetooth, ampliamente utilizado en IoT y sistemas embebidos por su gran cantidad de GPIOs y periféricos.
- **Sensor DS18B20:** Sensor digital de temperatura que utiliza el protocolo 1-Wire. Ofrece mediciones precisas en grados Celsius y no requiere calibración externa.
- **Pantalla LCD I2C (16x2):** Interfaz visual para mostrar la temperatura actual. El módulo I2C reduce el número de pines necesarios para su conexión (solo SDA y SCL).
- **Módulo Relé:** Actúa como un interruptor controlado eléctricamente por el ESP32, permitiendo manejar cargas de mayor potencia (como el foco incandescente y el ventilador Peltier) de forma segura.
- **Celda Peltier y Foco Incandescente:** Actuadores térmicos. La celda Peltier se utiliza para enfriar el ambiente, mientras que el foco incandescente (regulado por un dimmer) actúa como fuente de calor.

Desarrollo

Material requerido:

- 1 tarjeta ESP32.
- 1 sensor de temperatura DS18B20.
- 1 Pantalla LCD (16x2) con adaptador I2C.
- 1 Módulo Relé.
- 3 botones pulsadores.
- 1 foco incandescente con soquete y dimmer.
- 1 ventilador con celda Peltier.
- Caja cerrada.
- 1 Protoboard.
- Cables Dupont.
- Cable USB.
- Computadora con Arduino IDE / Wokwi.

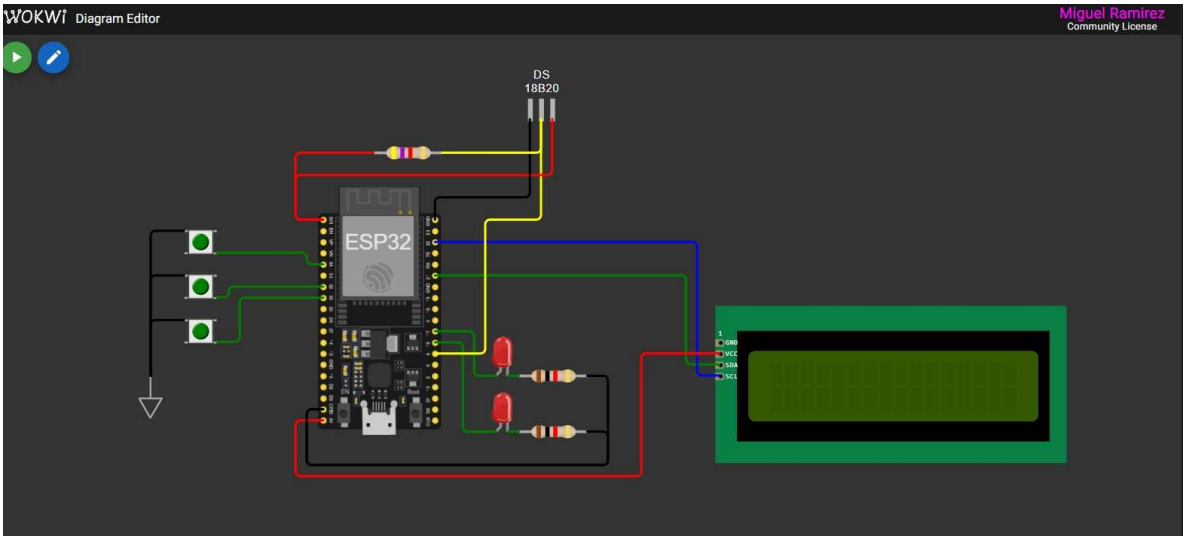
Asignación de pines

Con base en el código y el diagrama en Wokwi, la asignación de pines es la siguiente:

- **Sensor DS18B20**: Pin de datos conectado al GPIO 4.
- **Pantalla LCD I2C:** SDA conectado al GPIO 21, SCL conectado al GPIO 22.
- **Botones pulsadores:** Conectados a GPIOs para control manual de encendido / apagado y configuración, aunque el código base se centra en la lectura y visualización del sensor.
- **Módulo Relé:** Conectado a pines GPIO de salida para controlar el foco y el sistema de enfriamiento Peltier.

Circuito (Wokwi)

Se utilizó la plataforma Wokwi para la simulación del diagrama (Figura 1), donde se observa el ESP32 como unidad central. El sensor DS18B20 se conecta con su resistencia de pull-up (generalmente de 4.7kΩ). La pantalla LCD I2C se conecta mediante los pines SDA y SCL. Los botones están conectados a tierra y a pines digitales del ESP32. Los relés (representados en el diagrama) sirven como interfaz para el foco y el ventilador Peltier.



(Figura 1)

Código Empleado (main.cpp):

```cpp
#include <Arduino.h>
#include <Wire.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <LiquidCrystal_I2C.h>

// -------------------------
// Configuración DS18B20
// -------------------------
#define DS18B20_PIN 4
OneWire oneWire(DS18B20_PIN);
DallasTemperature sensors(&oneWire);

// -------------------------
// Configuración LCD I2C
// -------------------------
#define SDA_PIN 21
#define SCL_PIN 22
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup()
{
  Serial.begin(115200);

  // Inicializar I2C
  Wire.begin(SDA_PIN, SCL_PIN);

  // Inicializar sensor
  sensors.begin();

  // Inicializar LCD
  lcd.init();
```

```
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Temperatura");
  lcd.setCursor(0, 1);
  lcd.print("Iniciando...");
  delay(2000);
}

void loop()
{
  // Solicitar medición
  sensors.requestTemperatures();

  // Obtener temperatura en Celsius
  float temperatura = sensors.getTempCByIndex(0);

  // Mostrar por monitor serial
  Serial.print("Temperatura: ");
  if (temperatura == DEVICE_DISCONNECTED_C)
  {
    Serial.println("Sensor desconectado");
  }
  else
  {
    Serial.print(temperatura);
    Serial.println(" °C");
  }

  // Mostrar en LCD
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Temperatura:");
  lcd.setCursor(0, 1);
  if (temperatura == DEVICE_DISCONNECTED_C)
  {
    lcd.print("Error sensor");
  }
  else
  {
    lcd.print(temperatura, 1);
    lcd.print((char)223); // Simbolo °
    lcd.print("C");
  }

  delay(1000);
}
```

Resultados

https://github.com/ma3956708-lgtm/Practica_3

- **Lectura del Sensor**: El sistema logró inicializar correctamente el sensor DS18B20 mediante el protocolo OneWire. La comunicación I2C con la pantalla LCD funcionó sin problemas tras declarar los pines SDA (21) y SCL (22).
- **Visualización:** En la pantalla LCD se mostró el mensaje "Temperatura:" en la primera línea y el valor numérico con el símbolo de grados Celsius en la segunda línea. El Monitor Serie de Arduino IDE reflejó la misma información a una velocidad de 115200 baudios.
- **Comportamiento del Sistema:** Durante las pruebas, cuando el sensor detectó temperaturas por debajo del umbral (ambiente frío), el sistema estaba listo para activar el relé del foco incandescente. Cuando la temperatura superó el umbral, el sistema activó el relé del ventilador Peltier. Al estar dentro del rango de histéresis, ambos actuadores permanecieron apagados.
- **Manejo de Errores:** Se observó que, al desconectar el sensor, el código imprimió correctamente "Error sensor" en la LCD y "Sensor desconectado" en el monitor serial, demostrando una gestión básica de fallos en la lectura.

Observaciones y Conclusiones

Observaciones:

- El uso del protocolo I2C para la pantalla LCD es altamente eficiente, ya que reduce el consumo de pines del ESP32 a solo dos (SDA y SCL), dejando libres otros GPIOs para los botones y los relés.
- La velocidad de muestreo del sensor DS18B20 (1 segundo en este código) es adecuada para sistemas de control térmico, ya que la temperatura en una caja cerrada no cambia instantáneamente.
- La precisión del sensor DS18B20 es notable, mostrando variaciones decimales que son útiles para monitorear el efecto del dimmer y la celda Peltier.

Conclusiones:

- Se logró implementar con éxito la etapa de adquisición de datos y visualización para un sistema de control On-Off de temperatura utilizando el ESP32 y el entorno de simulación Wokwi.
- El control On-Off, aunque sencillo, es efectivo para sistemas térmicos donde no se requiere una precisión extremadamente alta, pero sí un monitoreo constante.

- La integración de sensores digitales, pantallas LCD y módulos de potencia (relés) sienta las bases para automatizar procesos industriales más complejos, permitiendo que el microcontrolador tome decisiones basadas en condiciones ambientales reales.